Heron Simulator Architecture

The simulator owns Gazebo worlds, Heron spawn configuration, synthetic sensor providers, scenario entities, timing adapters, the drive plant, and hydrodynamic overrides. GRANDE composes the scenario; MARINER owns estimator, map, planner, and accepted drive; ORACLE owns mission semantics.

Gazebo ground truth supports sensor generation and labelled evaluation. It is not substituted for MARINER estimator feedback in navigation or control. Synthetic topics identify source and calibration eligibility.

Scenario YAML carries simulator world, spawn, entity, and map-bound facts consumed by ORACLE through GRANDE. That is integration convenience, not simulator ownership of mission policy.

Scenario entity YAML and Gazebo geometry duplicate parts of the same scene, and no generator proves parity. Evaluation manifests identify both inputs so a mismatch remains an authoring defect rather than hidden uncertainty.

Heron Simulator Operation

The simulator normally starts through GRANDE:

```
cd ~/catkin_ws/heron_ws/src/grande/grande
python3 run.py bringup --mode sim --scenario harbor
```

The runner uses an isolated ROS master, normally port `11312`. Status, goal, and cancellation commands select that graph explicitly.

`heron_world.launch` selects world, rendering, time, and profile environment. `spawn_heron.launch` creates the vehicle and attaches sensor, timing, telemetry, and propulsion providers. GUI, headless, RViz, and software-rendering options change display behavior but not state or command meaning.

Simulated messages and propulsion command timeout use ROS simulation time. MARINER uses ROS time for selected simulation control loops and monotonic time on hardware; some cancellation barriers use wall timers. Runner process supervision and shutdown use wall time and remain scoped to child processes and the isolated master it started.

`open_water.world` references external `ned_frame` and `sand_heightmap` models. Runtime output retains scenario, configuration, commit, and synthetic-source provenance.

Heron Simulator Sensors

The simulator publishes device-like observations for GRANDE sensor classes. The goal is interface and timing fidelity within a declared synthetic model, not a claim that Gazebo reproduces full physical error distributions.

`urdf/sensors.urdf.xacro` defines simulated sensor additions and imports the canonical sensor poses from IG Handle. Frame names, geometry, and topic meanings therefore remain aligned with the physical sensor contract. Gazebo measurement, noise, rendering, and transport behavior are simulator-owned artifacts.

Gazebo plugins provide LiDAR, camera, IMU, and vehicle observations. `sim_ig_timing.py` publishes IG Handle-style timing surfaces. `sim_sense.py` publishes explicitly synthetic Heron telemetry tied to the selected propulsion plant.

Multibeam and Ping360 providers translate Gazebo rays into their canonical profile meanings. They represent geometry and transport, not acoustic propagation, transducer response, multipath, turbidity, or real latency.

Ground truth drives synthetic providers, but navigation consumes MARINER's estimator surface. Synthetic provenance remains attached even when downstream interfaces match physical acquisition.

Heron Simulator Scenarios

Scenarios bind a Gazebo environment, initial vehicle state, semantic entities, and integration settings into a reproducible configuration.

The tank world centers its tank, water-surface, and target models on the spawn/map origin so symmetric positive and negative navigation coordinates stay inside the controlled water volume. The harbor profile supports mapping, navigation, exploration, and inspection. Open water references external `ned_frame` and `sand_heightmap` models.

The exploration arena is a synthetic, logically bounded, reduced-load frontier integration scenario. It uses three finite static structures: `exploration_wall`, `exploration_return_wall`, and `exploration_south_breakwater`. They provide observable planar geometry and a bounded search area while ORACLE discovers frontiers from a fresh live map. Entity YAML remains simulator/evaluation reference truth in mapless runs rather than a planner-visible semantic prior. Success in this arena exercises software interfaces and the configured simulator plant; it does not validate physical sensor error, hydrodynamics, or vehicle performance.

Scenario YAML supplies initial pose, entities, world selection, offsets, and environmental map bounds. Mission and navigation policy comes from named GRANDE runtime profiles rather than simulator configuration.

Semantic records describe what ORACLE may discover; Gazebo files describe collision/visual geometry. No generator currently guarantees parity. Recorded runs identify both scenario and world inputs.

Scenario map bounds flow to ORACLE as environmental input. A result is meaningful only with its world, vehicle profile, sensor configuration, initialization, controller, mission profile, and repository state.

Heron Simulator Propulsion

The propulsion path converts MARINER's normalized left/right drive request into Gazebo forces and synthetic actuator telemetry. It is a simulator plant, not a physical-Heron calibration.

`drive_to_thrusters.py` represents direction, deadband, saturation, voltage scaling, lag, slew, and reversal blanking and publishes synthetic PWM, RPM, current, voltage, thrust, and status. `sim_sense.py` uses the same plant state so electrical surfaces remain internally consistent.

The retained S4.8 and S4.9 qualifications used the standard `config/thruster_dynamics.yaml` plant with the optional empirical actuator proxy disabled. Both selected MARINER's force-domain cascade and identity current stage. The four-regime inverse upstream and the simulator plant downstream are separate models: agreement within this loop is software behavior, not an independent force measurement.

Vehicle mass, inertia, fluid density, damping, and thruster placement determine motion response. Overlays are part of recorded simulator configuration and do not change the real Heron path.

The empirical proxy module validates/interpolates an optional proxy before plant use. Those fail-closed checks are runtime safety. Command-to-current, current-to-thrust, thrust-to-motion, and closed-loop control remain separate relationships. Simulation can expose asymmetry, reversal, saturation, timing, and fallback but cannot establish the physical force law.

The defensible description is **physics-based simulation with provisional, partly data-informed actuator parameterization**. Historical data informed some electrical/asymmetry choices, but mass, inertia, damping, absolute force, and water-relative disturbance are not jointly identified or physically validated. Accordingly, this simulator is the most integrated maintained software plant, not an identified data-based model of the real Heron.